Practical Assessment: Three-Tier AWS Web Application

Associate Cloud Engineer | Cloud Platform Team | Synergy Maritime Pvt Ltd

Format	Hands-on build in a sandbox AWS account, then a live review with the interviewers.
Time	4 hours of build time, plus a 45 minute review. Only the core tasks are required. Additional tasks are marked [OPTIONAL].
Tools	Console, AWS CLI, Terraform or CloudFormation, your choice. AI tools are allowed (see section 2).
Application	Docker's public **awesome-compose** repository, sample `nginx-flask-mysql`. Your interviewer gives you the exact commit.
Access	Sandbox account details, region and limits are shared separately by your interviewer.

Core and optional. Everything not tagged is **core** and expected. Anything tagged **[OPTIONAL]** is an additional task. If you do not build it, describe in your hand-over what you would do and how. Skipping an optional task is not penalised.

1. Scenario

Your team is moving a small web application onto AWS. Build it as a three-tier environment: a load balancer in public subnets, web and application containers on a Linux VM in a private subnet, and a managed database in isolated private subnets. Set up logging and basic monitoring, prove it works end to end, and leave a short runbook.

The project

Clone `https://github.com/docker/awesome-compose/tree/master/nginx-flask-mysql`. It has three services: an nginx proxy, a Flask backend and a database. The home page renders blog posts read from the database, so a working page proves the app can reach its DB.

- Remove the bundled database service and point the backend at **RDS** (MySQL or MariaDB engine).
- Check how the backend takes its DB host, user, database name and password (some may be hard-coded) and supply them from configuration, not from values committed to the repository.
- You are not expected to change application logic beyond configuration. Make any small change needed (for example a hard-coded host) and note it in your design notes.

2. Ground rules

AI tools are allowed, and we assume you will use them. We are not testing syntax recall or typing speed. We are testing whether you can design this sensibly, check that what you built is correct and secure, and diagnose it when something breaks. You are responsible for everything you run and submit, and you should expect to explain any setting, policy or command in your solution.

- Work only inside the sandbox account, region and limits in your access details. The account has guardrails. If an action is denied, read the error and work out why. Do not try to bypass the guardrails.
- Never paste sandbox credentials, access keys or secrets into an AI tool, a chat, or a public repository. Describe the problem instead and redact account IDs and keys.
- **Prioritise.** If time runs short, finish in this order: (1) network, security group chain, containers running, ALB to app to RDS working; (2) CloudWatch logs and one alarm; (3) runbook. Tell us what you skipped and why.
- Ask us if the brief is unclear. Asking a good question is not penalised.

3. Target architecture

VPC across 2 Availability Zones: **Public subnets** (ALB, NAT gateway) -> **Private app subnets** (one Linux EC2 VM running web + app containers) -> **Private DB subnets** (RDS, no public access). The VM reaches Git and Docker Hub outbound only, through the NAT gateway. Supporting services: CloudWatch (logs, alarms) and SNS for email notifications. Secrets Manager or SSM Parameter Store is optional.

The design shows intent, not every resource. Subnet counts, sizing and wiring are your decisions.

4. What to build

Stage 1: Network, security groups and database

- One VPC across two AZs with public, private app and private DB subnets (two of each is enough).
- Public route table to an internet gateway. Private app route table to one NAT gateway. The DB route table has no internet route.
- No EC2 instance or database may have a public IP address.
- Security groups: ALB accepts HTTP from the internet. App tier accepts the app port from the ALB security group only. DB tier accepts the database port from the app security group only. Reference security groups, not CIDRs, between tiers.
- RDS (MySQL or MariaDB), smallest allowed class, in a DB subnet group of private DB subnets. Not publicly accessible, encryption at rest on, automated backups on. Single-AZ is fine.
- **[OPTIONAL]** Tight outbound rules on the app and DB security groups, with a reason for each rule.

> **Hint**
> - RDS takes 10 to 15 minutes to create. Start it as early as you can.

Stage 2: Application tier

- One Linux EC2 instance in a private app subnet with Docker and Docker Compose. Allowed instance types are in your access details.
- Run two containers from the sample's Compose setup: **web** (nginx proxy) and **app** (Flask, built from the sample's Dockerfile). Only the web container's port is exposed to the ALB.
- Containers restart automatically (restart policy, Docker enabled at boot) and the target group health check shows healthy targets. A 200 on the home page is enough.
- DB credentials must not be committed to Git or baked into an image layer. Pick **one** of the two methods below and be ready to explain how it does or does not expose the password on the host.
- Admin access is through SSM Session Manager, with no inbound SSH.

Credential method	How it works	Status
Simple	Create a root-only file (for example `/opt/app/db-password`) by hand over Session Manager, outside the repo, or use an untracked `.env` file. Supply it to the app container.	Core
Secrets Manager or SSM SecureString	Store the credentials in the service and fetch them at runtime with the instance role. No secret value in Git, image, Compose file or user data.	**OPTIONAL**

- **[OPTIONAL]** Fetch the secret at runtime with the instance role, with IAM scoped to that one secret or parameter.
- **[OPTIONAL]** Run containers as non-root where practical.
- **[OPTIONAL]** Container health checks in Compose.
- **[OPTIONAL]** Full automatic recovery after a reboot, including any secret fetch step.

> **Hint**
> - Read the backend source first to see which DB settings are hard-coded and how the password is read.

Stage 3: Monitoring

- **Logs:** web and app container logs go to CloudWatch Logs, with a retention period set on every log group.
- **Alarm:** one alarm that emails you through SNS, on the ALB or targets (5xx or unhealthy hosts). Be ready to say why you chose it and what you would do when it fires.
- **[OPTIONAL]** CloudWatch agent publishing memory and disk usage from the VM.
- **[OPTIONAL]** A second alarm on an agent metric (memory or disk) or on RDS.
- **[OPTIONAL]** One small dashboard (ALB requests and 5xx, healthy targets, VM memory, RDS CPU).

IAM

- An EC2 instance role with only what the VM needs (Session Manager and the CloudWatch log groups). No access keys, no administrator policies, no wildcard action on wildcard resource.

- Follow any role naming or permissions-boundary conventions in your access details.
- **[OPTIONAL]** Scope permissions to the one secret and the specific log groups (and the agent's metric namespace if used).

5. Prove it works

Collect evidence for each check (command output or screenshots) and include it in your hand-over.

#	Check
1	The app loads in a browser through the ALB DNS name, the target group shows healthy targets, and the page shows data read from RDS.
2	Both containers are running with a restart policy. Reboot the VM and show they come back. Show where the DB password comes from and prove it is not in Git, the image or the Compose file.
3	From the internet you cannot reach the VM or the database. The database accepts connections only from the app tier.
4	Container logs from both containers are in CloudWatch Logs with retention set. **[OPTIONAL]** Agent metrics and a dashboard.
5	Stop the app container deliberately. The alarm fires, the email arrives, and you find the cause from the logs. Then restore service.

6. What to hand over

- Architecture diagram and evidence for the five checks.
- A one-page runbook (deploy, health checks, logs, likely failures, teardown) and a few lines of design notes on your trade-offs.
- Your Dockerfile, Compose file and scripts or templates, with a short note on where you used AI and what you corrected.
- For each [OPTIONAL] task you did not build, a short note on what you would do. Confirm that everything is torn down.

7. How we assess and the review

We look at design judgment, security, verification and troubleshooting more than a perfect build. Specifically:

- **Architecture:** correct tiering, a clean security group chain, an isolated database, and a reason for each choice.
- **Security:** credentials kept out of Git and images, no public exposure, and least-privilege IAM.
- **Verification:** evidence that you tested the system and did not simply trust generated output.
- **Observability:** useful logs and an alarm that you proved fires.
- **Operations:** a runbook someone else can follow, honest design notes, and a clean teardown.
- **Optional tasks** earn extra credit and are never held against you.

After the build you walk us through the architecture and evidence, explain your key decisions, and take part in a short live troubleshooting exercise. We may change something in your environment and ask you to find and fix it. We will also ask how you would operate this in production. There are no trick questions. We want to see how you reason when you cannot simply ask an assistant for the next step.

If you hit a problem with access or the sandbox itself, tell your interviewer straight away. That time will not count against you.